Advanced C# Tasks - Day 2: Generics & Collections

Task 1: Generics and Constraints

Goal: Explore Generics and Constraints in C#

Step 1.1: Create the Animal Hierarchy

- Define a base class Animal with an Age property and a Name property.

- Create two subclasses: Lion and Sparrow.

- The Twist: Give the Lion a unique method called Roar() and the Sparrow a unique method called Fly().

Step1.2: Create a Generic Cage

- Create a class Cage<T>.

- Use a Generic Constraint to ensure T can only be an Animal (use where T : Animal).

- Inside Cage<T>, add a method void Arrive(T resident).

- In this method, if the animal's Age is greater than 8, throw a custom InvalidAgeException.

Step 1.3: Demonstrate Type Safety

- In your Main method, instantiate a Cage<Lion>.

- Try to add a Sparrow to the Cage<Lion>.

- Observation: Notice the compiler stops you immediately. This is why we use Generics to prevent "biological disasters" at compile-time rather than runtime!

- Retrieve the Lion from the cage and call Roar().

- Observation: Notice you don’t have to "cast" the animal. The computer already knows it’s a Lion.

Task 2: Collections

Goal: Explore and manipulate collections in C# like arrays, lists, and dictionaries.

Step 2.1: Create a Student Grades Tracker using a Dictionary<string, List<int>>.

- Use the student's name as the **Key** and a list of their grades as the **Value**.

- Implement functionality to add new grades to an existing student and display the average grade for every student in the system.

- Write logic to find and display the name of the student with the highest average grade.

Step 2.2: Build a ToDo List Application using a Queue<string>.

- Implement features to add a new task and display all current tasks along with the total count (e.g., "You have 5 tasks").

- Add a feature to "Mark as Completed" which removes a specific task from the list.

Step 2.3: Create a Dictionary where the contact names are keys and phone numbers are values.
- Implement a method to search for a contact using the contact name. If the contact does not exist, throw an exception or return a default message. Make sure this logic is customized.

Step 2.4: Implement a Stack<T> for managing browser history.
- Push visited pages to the stack and allow going back to previous pages using Pop (LIFO).
- Each page visited should be stored in the stack.